1)Write a C program that adds two integer arrays, A and B, each containing N elements, and stores the result in array C.

CODE:

```
#include <stdio.h>
#include <omp.h>

int main()
{
   int N;

   printf("Enter the number of elements: ");
   scanf("%d", &N);

   int A[N], B[N], C[N];

   printf("Enter elements of array A:\n");
   for (int i = 0; i < N; i++)
      scanf("%d", &A[i]);

   printf("Enter elements of array B:\n");
   for (int i = 0; i < N; i++)
      scanf("%d", &B[i]);

   // Parallel addition
   #pragma omp parallel for
   for (int i = 0; i < N; i++)
   {
      C[i] = A[i] + B[i];
   }

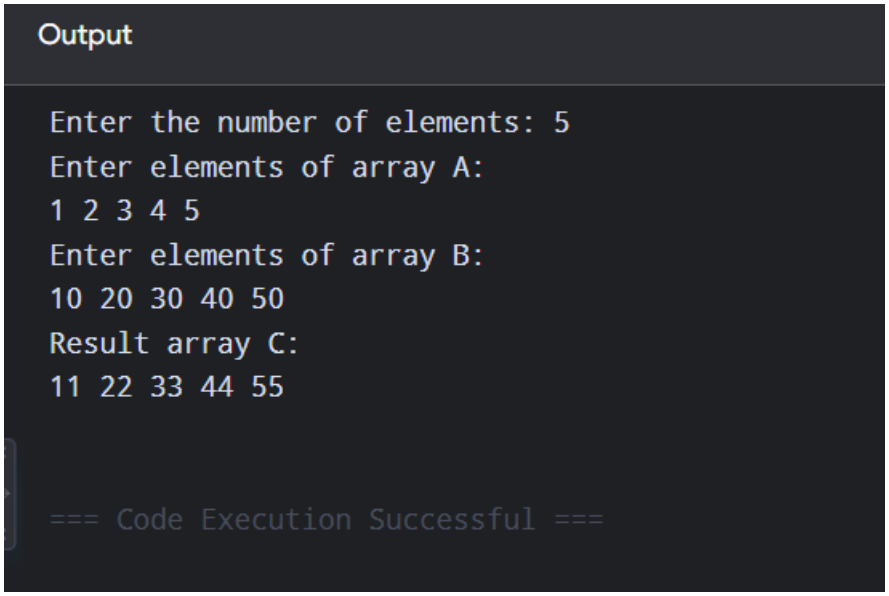
   printf("Result array C:\n");
```

```
    for (int i = 0; i < N; i++)
        printf("%d ", C[i]);

    printf("\n");

    return 0;
}
```

OUTPUT:



2) Write a C program that calculates the sum and average of N floating-point numbers stored in an array.

CODE:

```
#include <stdio.h>
#include <omp.h>

int main()
{
    int N;
```

```
    printf("Enter the number of elements: ");
    scanf("%d", &N);

    float A[N];

    printf("Enter %d floating-point numbers:\n", N);
    for (int i = 0; i < N; i++)
        scanf("%f", &A[i]);

    float sum = 0.0;

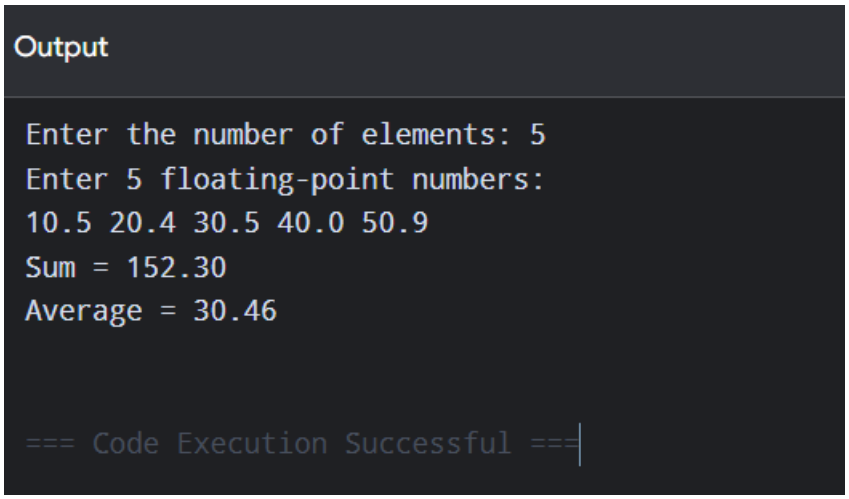
    // Parallel sum using reduction
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < N; i++)
    {
        sum += A[i];
    }

    float average = sum / N;

    printf("Sum = %.2f\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

OUTPUT:



3) Write a C program that finds the largest value in an integer array and reports its index.

CODE:

```
#include <stdio.h>
#include <omp.h>

int main()
{
   int N;

   printf("Enter the number of elements: ");
   scanf("%d", &N);

   int A[N];

   printf("Enter %d integers:\n", N);
   for (int i = 0; i < N; i++)
      scanf("%d", &A[i]);
```

```
int max = A[0];
int index = 0;

#pragma omp parallel
{
   int local_max = A[0];
   int local_index = 0;

   #pragma omp for
   for (int i = 0; i < N; i++)
   {
      if (A[i] > local_max)
      {
         local_max = A[i];
         local_index = i;
      }
   }

   // Combine results safely
   #pragma omp critical
   {
      if (local_max > max)
      {
         max = local_max;
         index = local_index;
      }
   }
}

printf("Largest value = %d\n", max);
printf("Index = %d\n", index);
```

```
    return 0;
}
```

OUTPUT:

